Prahari — Complete Technical Documentation

Privileged-Access Insider-Threat Detection & Management for Banks

FinSpark'26 · Bank of Maharashtra · Problem Statement 1 — Privileged Access Misuse & Insider Thr

Repository: github.com/positromen/FinSpark-26-Trial

Contents

1 Executive Summary

Prahari ("sentinel") is a Privileged Access Management (PAM) *and* insider-threat detection platform for banks. It sits between privileged staff — DBAs, sysadmins, network and application admins, and vendors — and critical systems, and does four things for every privileged session:

- **Watches** every privileged action as a normalized, **recorded** session with a replayable command trail.
- **Scores** each session **0–100 in real time**, fusing a rule engine with AI behavioural analytics (UEBA) and peer comparison, always with a human-readable *why*.
- **Responds** adaptively — **ALLOW / STEP-UP MFA / MAKER-CHECKER / BLOCK** — tagged with the detected **insider type**.
- **Protects** its own credentials and audit log with **NIST post-quantum cryptography** (ML-KEM-768 vault and ML-DSA-65 signed hash-chain).

It ships as a real two-sided product: an **Employee Portal** where staff act and enforcement happens live, and a **SOC Console** where analysts watch, replay sessions, review access, and verify the audit chain. It runs **fully offline** on SQLite and is one connection string away from PostgreSQL.

All six judged outcomes are delivered: detect misuse of privileged accounts; identify insider threats in real time; AI-driven behavioural analysis; risk-based access control; protect critical administrative systems; and quantum-proof cryptography for credentials and audit artefacts.

2 Problem and Context

Traditional bank security — firewalls, antivirus, perimeter IAM — is built for **outsiders**. It is largely blind to a **trusted insider** who already holds privileged access. Problem Statement 1 names three insider archetypes explicitly, all of which must be demonstrable: **malicious**, **negligent**, and **compromised**.

Common failure modes in privileged access:

- **Dormant and vendor accounts** never deactivated — a classic entry point.
- **Privilege escalation** performed outside the grant process.
- **Mass export** of customer records; **after-hours** bulk access.
- **Expired vendor access** still in daily use (negligence).
- **Account takeover** — new geography or device, inhuman action pace (compromise).
- **Tamperable audit logs** — the insider edits the evidence.
- **"Harvest-now, decrypt-later"** — credentials and evidence encrypted with classical cryptography today may be broken by future quantum computers.

Why we chose it. It carries the highest business impact for a bank — fraud loss, RBI compliance, and customer trust — and it lets us combine real-time AI/UEBA, PAM controls, and post-quantum cryptography into one coherent, defensible solution.

3 Solution Overview

Prahari performs four steps, in order, for every privileged session: record and normalize the activity, score it, enforce an adaptive response, and protect the resulting evidence with post-quantum cryptography. Two role-based experiences sit behind one login:

- **Employee Portal** — a privileged-access console where staff perform actions (query, file access, configuration change, privilege escalation, bulk export). Each action is scored and **enforced live** before it takes effect.

- **SOC Console** — the analyst's control room: live sessions, a risk gauge, insider-type-tagged alerts, a why-flagged panel, a session timeline, **privileged-session replay**, a risk heatmap, a **PAM access review**, and post-quantum audit **verify / tamper** controls.

4 System Architecture

Six modules carry activity from ingestion through analytics, scoring, response, and the post-quantum security layer.

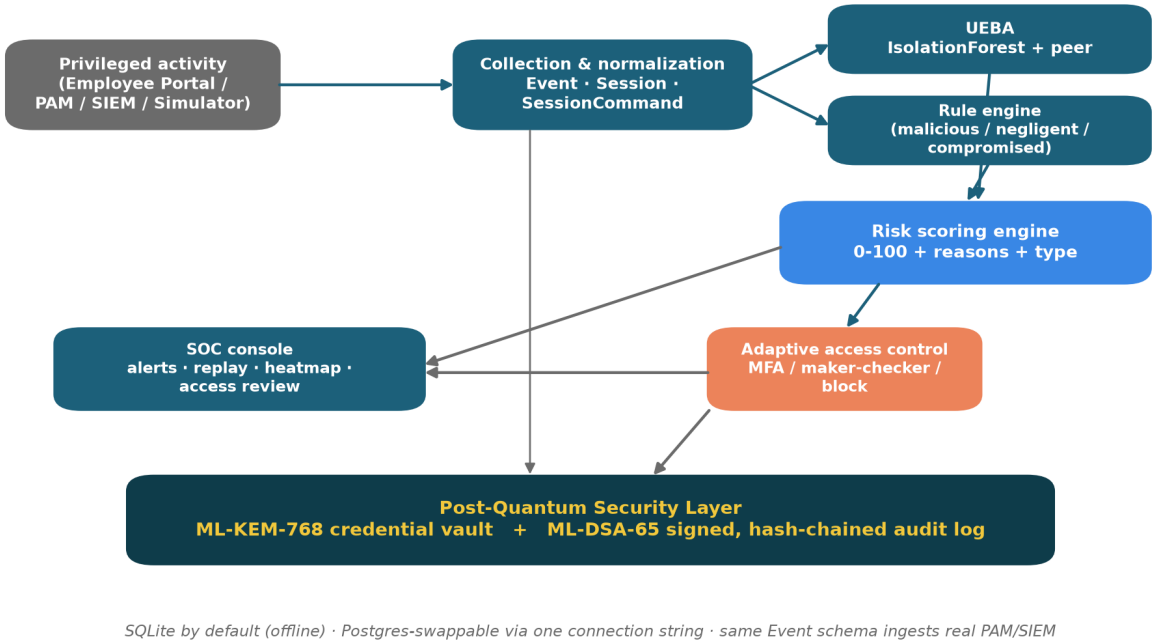


Figure 1: Prahari system architecture — six modules from ingestion to the post-quantum security layer.

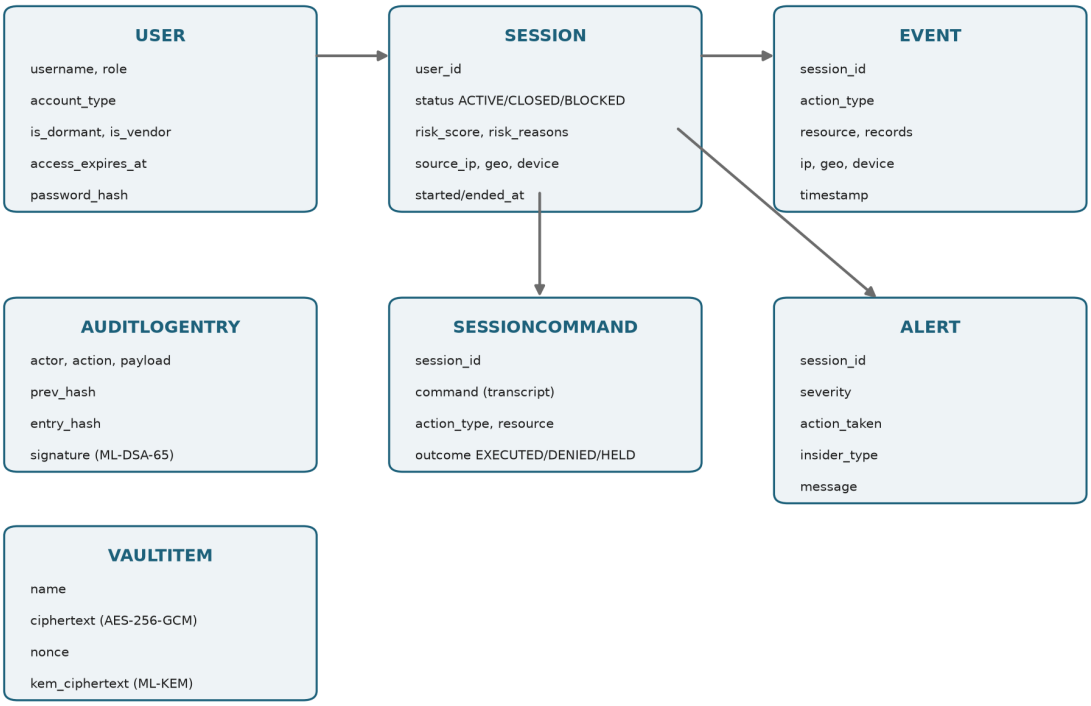
The Employee Portal, and in production any PAM/SIEM feed, map onto the same normalized `Event` schema; the built-in simulator produces the same shape for the demo.

Technology stack

Layer	Choice
Backend / API	Python 3.14, FastAPI + Uvicorn
Storage	SQLAlchemy ORM · SQLite (default) -> PostgreSQL-swappable
AI / UEBA	scikit-learn IsolationForest + peer baselining
Post-quantum crypto	liboqs-python — ML-KEM-768 (FIPS 203), ML-DSA-65 (FIPS 204); AES-256-GCM
Authentication	PBKDF2-HMAC-SHA256 passwords + HMAC-signed tokens (standard library)
Real-time	FastAPI WebSocket
Frontend	React 19 + Vite + Tailwind 4 (+ Recharts)
Packaging	Docker + docker-compose · run.ps1 / run.sh
Testing	pytest (31 tests)

5 Data Model

Seven ORM entities capture the workforce, their recorded sessions and actions, the detection output, and the post-quantum artefacts.



One USER → many SESSIONs → many EVENTs; each SESSION carries a recorded command trail (SessionCommand); alerts, audit entries and vault items complete the model.

Figure 2: Entity model — users, sessions, events, recorded commands, alerts, audit entries and vault items.

`SessionCommand` is the **privileged-session recording**: every action writes a realistic command line (for example, `psql core-banking-db -c "COPY customers TO '/tmp/out.csv' CSV;" -- 5000 rows`) with an outcome, so a session replays like a terminal transcript.

Seeded cast (all password `prahari123`):

Username	Role	Notes
rmehta, spatil	DBA	permanent staff
akulkarni, pjoshi	SYSADMIN	permanent staff (akulkarni = compromised-scenario subject)
vdeshmukh	NET_ADMIN	permanent staff
nshinde	APP_ADMIN	permanent staff
ext_dsouza	CONTRACTOR	dormant vendor, access expired 120 days -> malicious attacker
ext_rao	CONTRACTOR	active vendor, access expired 18 days -> negligent subject
soc_admin	SOC_ANALYST	logs into the SOC console (account_type = ANALYST)

6 Detection Engine

Two detectors feed one scorer.

6.1 Rule engine

Each rule returns a **reason**, a **weight**, and an **insider_type** tag.

Rule	Fires when	Weight	Type
DORMANT_REACTIVATION	a dormant account logs in	30	malicious
PRIVILEGE_ESCALATION	a privilege-change event occurs	25	malicious
AFTER_HOURS_ACCESS	activity between 00:00 and 06:00	20	malicious
MASS_EXPORT	at least 1000 records in one session	30	malicious
NO_BUSINESS_RELATIONSHIP	resource outside the user's role	15	malicious
NEW_GEO	login location is not the home location	16	compromised
NEW_DEVICE	unrecognized device **with** a foreign location	12	compromised
ATYPICAL_HOUR	login in an off-shift hour (06, 20–23)	8	compromised
RAPID_FIRE	at least 5 actions within 180 seconds (inhuman pace)	8	compromised
EXPIRED_ACCESS_IN_USE	the user's access grant has lapsed	30	negligent
UNMANAGED_DEVICE	sensitive data from a new device at the **home** location	30	negligent

The **dominant insider type** of a session is the category carrying the most rule weight (tie-break priority: malicious, then compromised, then negligent).

Device disambiguation. A new device *with* a foreign geography reads as account takeover (`NEW_DEVICE`, compromised); a new device from the *home* location reads as a personal, unmanaged laptop (`UNMANAGED_DEVICE`, negligent). This keeps the two families cleanly separated.

6.2 UEBA — behavioural analytics

Every session becomes a 7-feature vector: `login_hour, event_count, total_records, distinct_resources, config_changes, offsite_ip, new_device`.

- An **IsolationForest** (100 trees) is trained on **closed historical sessions**, including **cumulative prefixes** of each session — so live sessions (which arrive one action at a time) are in-distribution and normal early activity is not flagged merely for being short.
- The raw anomaly is normalized to **0–100** against the baseline distribution (median maps to 0, first percentile maps to 100).
- **Peer comparison** measures the session's record volume against the same-role average — for example, "5000× more records than CONTRACTOR peers".

6.3 Risk scoring

The final score fuses the two detectors:

$$\text{score} = \min\big(\text{rule\_weight (capped at 80)} + 0.25 \times \text{UEBA\_anomaly} + \text{peer\_bonus (10 if} \geq 5 \times \text{peers)},\ 100\big)$$

UEBA is a **secondary nudge** (at most 25 points): the stable, explainable rule engine sets the band; the behavioural model refines within it. Every score ships a reason list (the "why-flagged" panel) and the insider type.

Figure 3: Scoring pipeline — rules and UEBA fuse into a 0–100 score that maps onto the response ladder.

7 The Three Insider Types

Three scripted scenarios (one SOC button each) prove three **distinct** response paths. The behaviour is verified live and by automated tests on an independent random seed.

Scenario	Dominant signals	Score band	Response
Malicious	dormant + escalation + mass-export + out-of-role + after-hours	>= 85	**BLOCK**
Compromised	new geo + new device + atypical hour + rapid-fire	40–69	**STEP-UP MFA**
Negligent	expired access + unmanaged device	70–84	**MAKER-CHECKER**

Type-aware policy. Negligence is a control failure to remediate with a **human second-check**, not an attack to hard-block — so a negligent session is **floored to maker-checker review** and **never escalated to an automated block**. Malicious blocks; compromised steps up. This is enforced in `decide(score, insider_type)`.

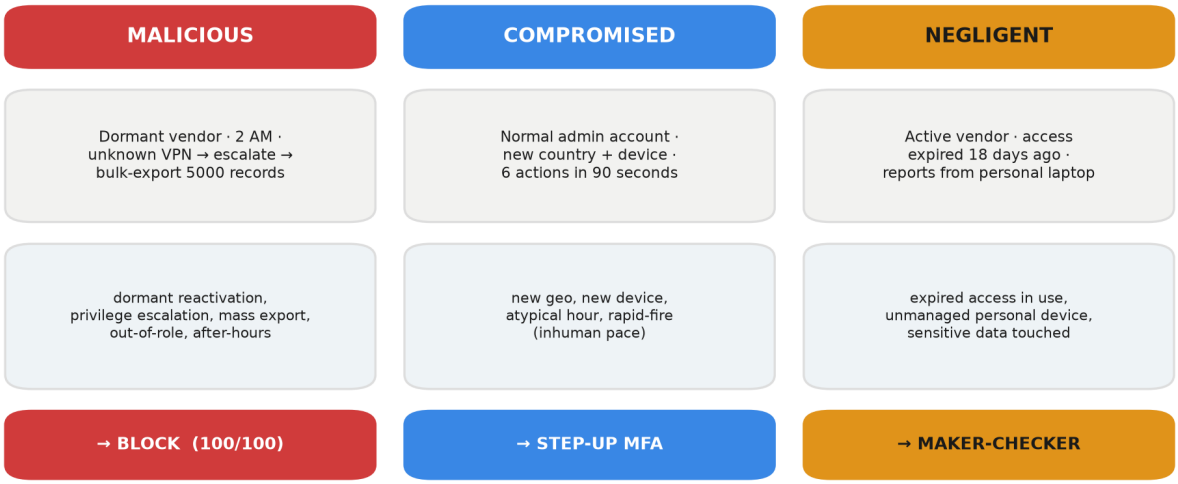


Figure 4: Three insider types — one engine, three distinct, correctly-typed responses.

8 Adaptive Response and PAM

8.1 Live enforcement

Each portal action re-scores the **whole session including the candidate action**, then enforces the decision **before the action takes effect**.

Key safety properties:

- **Blocked accounts stay locked.** A blocked session is returned on re-login; a blocked user cannot open a fresh session to continue.
- **No score-gaming.** Challenged actions (MFA or maker-checker) are **not persisted** until actually allowed, so retrying does not inflate the session.
- **Server-side enforcement.** The API refuses to execute; the UI overlay is cosmetic.

8.2 PAM surface

- **Privileged-session recording** (`GET /soc/sessions/{id}/commands`) — the replayable command trail; blocked commands show as DENIED (struck through), held ones as HELD.
- **Access review** (`GET /soc/access-review`) — every privileged account with standing-risk flags (DORMANT / VENDOR / EXPIRED) and a risk rating, surfacing lingering access *before* anything happens.
- **Maker-checker approval** (`POST /soc/sessions/{id}/approve`) — an analyst approves a held session; the approval is written to the signed audit log.

9 Post-Quantum Security Layer

All post-quantum operations sit behind one abstraction, `app/security/pqc.py`, exposing `kem_keypair`, `kem_encapsulate`, `kem_decapsulate`, `sign`, and `verify` over **ML-KEM-768** (FIPS 203) and **ML-DSA-65** (FIPS 204).

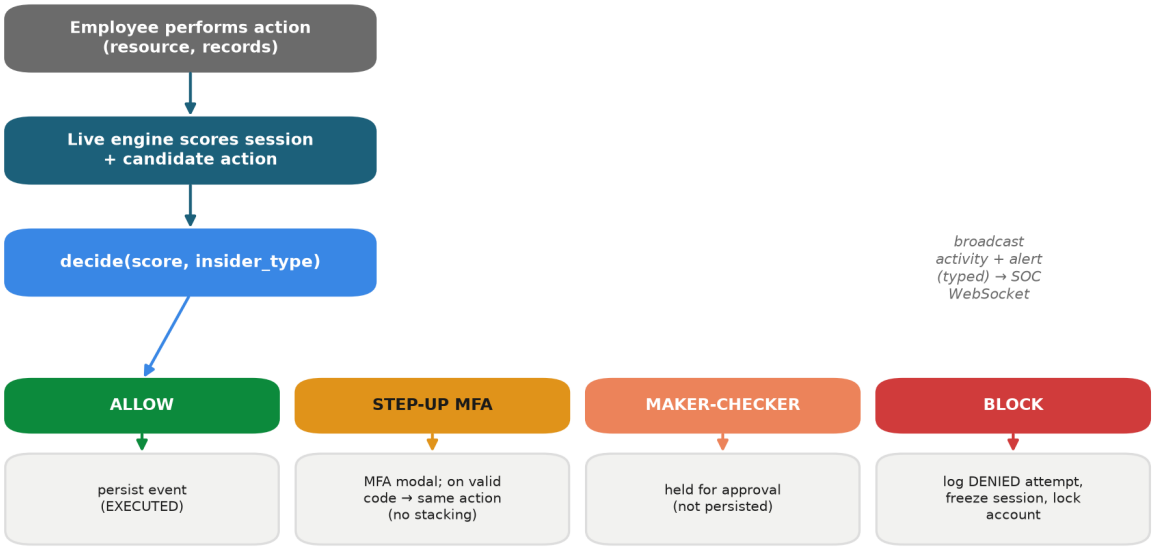


Figure 5: Live enforcement — every action is scored and routed to one of four responses before it takes effect.

9.1 Credential vault

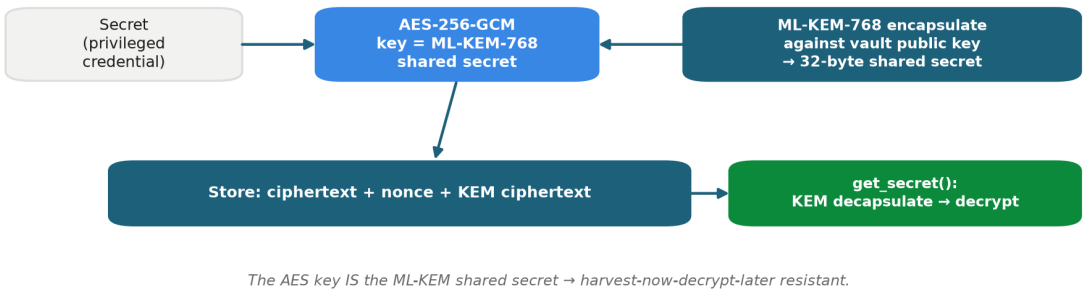


Figure 6: Credential vault — a secret is sealed with AES-256-GCM under an ML-KEM-768 shared secret.

The AES key **is** the ML-KEM shared secret; decryption requires the vault's KEM secret key. Data recorded today cannot be decrypted later, even by a quantum adversary.

9.2 Tamper-evident audit log

Each entry hash-chains the previous entry **and** is ML-DSA-65 signed. Editing any entry breaks its hash and every subsequent link; recomputing hashes is not enough, because the signature must still verify with the audit public key. `POST /demo/tamper` flips one record live, and `verify_chain()` then fails at the exact entry.

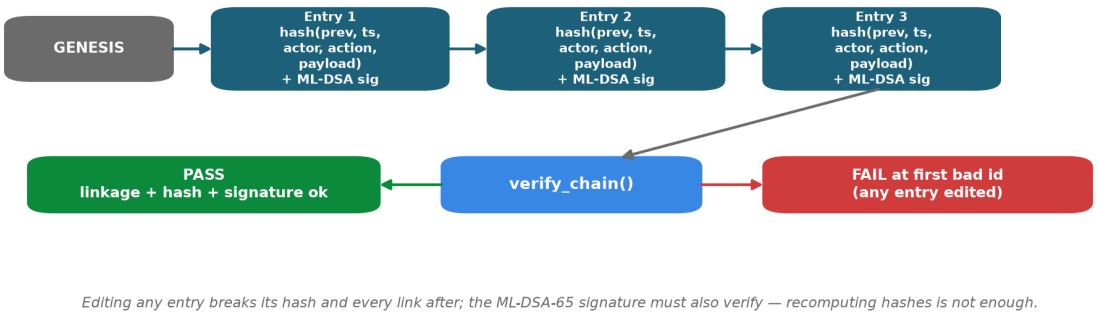


Figure 7: Audit chain — each entry hash-links the previous and is ML-DSA-65 signed; any edit fails verification.

10 Authentication and Authorization

- **Passwords** — PBKDF2-HMAC-SHA256 (200,000 rounds, per-user salt), in `app/security/auth.py`.
- **Tokens** — compact HMAC-SHA256-signed, JWT-like tokens with expiry, verified on every request.
- **Roles** — `EMPLOYEE` routes to the Employee Portal; `ANALYST` routes to the SOC Console. SOC endpoints are gated with `require_analyst` (HTTP 403 for employees). The WebSocket feed is broadcast-only.

This layer is demo-grade by design; a production deployment swaps in the bank's IdP/SSO behind the same dependency.

11 API Reference

Method	Endpoint	Auth	Purpose
POST	/auth/login	–	issue token (username / password)
GET	/auth/me	any	current identity
POST	/portal/bootstrap	employee	open live session + catalog + resources
POST	/portal/action	employee	perform action -> scored and enforced
POST	/portal/logout	employee	close session
GET	/soc/overview	analyst	users, scored history, heatmap, live sessions
GET	/soc/live	analyst	active / blocked sessions (typed)
GET	/soc/alerts	analyst	recent alerts (typed)
GET	/soc/access-review	analyst	PAM dormant / vendor / expired table
GET	/soc/sessions/{id}/commands	analyst	session recording replay
GET	/soc/sessions/{id}/events	analyst	session events
POST	/soc/sessions/{id}/approve	analyst	maker-checker approval
POST	/demo/scenario/{kind}	analyst	run scripted malicious / compromised / negligent

Method	Endpoint	Auth	Purpose
POST	/demo/tamper	analyst	edit an audit entry (proves detection)
GET	/audit, /audit/verify	analyst	list / verify the signed chain
GET	/pqc/info	any	active PQC provider and algorithms
POST, GET	/vault/secrets	analyst	store / retrieve a PQC-wrapped secret
WS	/ws/feed	–	live activity / alert / tamper frames
GET	/health	–	liveness

Interactive API docs are served at `/docs` (FastAPI / OpenAPI).

12 Frontend

React 19 + Vite + Tailwind 4, dark SOC theme, served by FastAPI from `frontend/dist` (same-origin, offline).

- **Login** (`pages/Login.jsx`) — role-routed, with a demo-account helper.
- **Employee Portal** (`pages/Portal.jsx`) — action console, connection badge, live risk gauge, activity timeline, MFA modal, maker-checker banner, and a full-screen BLOCK overlay.
- **SOC Console** (`pages/SocConsole.jsx`) — three scenario buttons, an audit banner, and panels: live sessions (typed), risk gauge, why-flagged, alerts (typed), timeline, session recording (terminal replay), heatmap, and access review.

The colour system reserves status colours (good / warning / serious / critical), colour-codes insider types (malicious red, compromised blue, negligent amber), uses tabular numerics, flashes on new alerts, and never encodes meaning by colour alone.

13 Security Considerations

- **Data minimization** — scores on privileged-activity metadata; no customer PII required.
- **Credential protection** — an ML-KEM-768-wrapped AES-256-GCM vault for privileged secrets.
- **Audit integrity** — hash-chained and ML-DSA-65-signed entries; tampering is cryptographically evident.
- **Enforcement** — server-side; blocked accounts stay locked; a challenged action is not persisted until actually allowed (no score-gaming, no bypass by re-login).
- **Authorization** — role-gated APIs; analyst-only SOC and PQC endpoints.
- **Offline by design** — no external calls at runtime; keys kept out of version control; `.env`-driven configuration; least-privilege demo accounts.
- **Explainability and compliance** — every decision has a plain-English reason, an immutable trail, and access reviews for standing, dormant and expired grants.

14 Scalability

- **Stateless scoring** — scale horizontally behind a load balancer; each request scores one session.
- **Storage** — SQLite to PostgreSQL by changing one connection string (pure ORM, no raw SQL).
- **Real-time fan-out** — WebSocket behind a Redis or Kafka broker to serve many SOC clients and high event throughput.

- **Ingestion** — generalizes to enterprise PAM/SIEM feeds (millions of privileged events per day) on the same `Event` schema.
- **Machine learning** — models retrain per role and shift nightly; a feature store holds baselines; the UEBA interface accepts an autoencoder upgrade with no application changes.
- **Post-quantum operations** — ML-KEM encapsulation and ML-DSA signing are sub-millisecond, negligible next to a database write.

15 Deployment and Running

```
# Windows PowerShell
.\run.ps1              # venv + deps + seeded DB + UI + API (offline)
.\run.ps1 -Reset       # wipe and reseed for a clean demo

# Git Bash / Linux / macOS
./run.sh               # same, POSIX

docker compose up --build      # container alternative
```

Then open `http://127.0.0.1:8000`. The first run on a fresh machine needs internet once (pip install plus a one-time liboqs build); afterwards it is fully offline. Run tests with `.venv/Scripts/python -m pytest`.

16 Testing

The suite contains **31 pytest tests**, including:

- **test_simulator** — user seeding, normal-day realism, determinism.
- **test_detection** — every rule, normal-versus-attack scoring, quiet-on-normal.
- **test_scenarios** — the three insider scenarios land on distinct, correct paths and the type-aware policy holds (negligence never auto-blocks), on an independent seed.
- **test_phase3** — attack block, alert, and the WebSocket loop.
- **test_portal** — authentication, role gating, live enforcement, MFA without event stacking, and blocked-stays-locked.
- **test_pqc** — KEM round-trip, sign/verify with forgery rejection, vault round-trip, and audit chain clean / tamper / signature-forgery.

17 Demo Walkthrough

Full narration is in `DEMO_SCRIPT.md`. In brief (two browser windows, side by side):

1. **SOC Console** (`soc_admin`) — quiet, with a green heatmap.
2. **Employee Portal** (`rmehta`) — a normal query is ALLOWED; an export of 1000 records triggers STEP-UP MFA (code `246810`).
3. **Attacker** (`ext_dsouza`) — escalate then export 5000 records -> full-screen BLOCK; the account is locked.
4. **SOC lights up** — a red live session at 100, a flashed CRITICAL alert tagged `malicious`, the why-flagged panel, the session replay (export struck through as DENIED), and a red heatmap cell.
5. **Two more insider types** — the SOC buttons run Compromised -> MFA and Negligent -> maker-checker, each correctly typed.
6. **PAM access review** — dormant, vendor and expired flags.
7. **Quantum-safe evidence** — Verify Chain (green) then Tamper (red, FAILED at the exact entry).

18 Roadmap

- **PS2 correlation** — link a privileged record-change to a suspicious downstream transaction.

- **Just-in-time access** and automated entitlement reviews.
- **Autoencoder** UEBA upgrade behind the same interface.
- **Enterprise integrations** — CyberArk / BeyondTrust PAM, Splunk / QRadar SIEM, IdP / SSO.
- **Workforce-wide insider-risk scoring** beyond privileged accounts.

19 Repository Map

```
PRAHARI/
  app/
    main.py               FastAPI app + lifespan + static UI mount
    config.py             pydantic settings (.env)
    pam.py                session-command recording + access review
    api/routes.py         REST + WebSocket endpoints
    api/ws.py             WebSocket broadcast manager
    detection/rules.py    rule engine (3 insider types, typed)
    detection/ueba.py     IsolationForest + peer baseline (prefix-trained)
    detection/score.py    0-100 risk score + reasons + insider type
    detection/response.py type-aware adaptive response
    detection/live.py     live per-action scoring & enforcement
    models/entities.py    ORM entities
    security/auth.py      PBKDF2 passwords + HMAC tokens
    security/pqc.py       ML-KEM-768 / ML-DSA-65 abstraction
    security/vault.py     quantum-safe credential vault
    security/audit.py     hash-chained + signed audit log
    simulator/            normal-day generator, 3 scenarios, seeder
  frontend/               React app (pages/ + components/) + prebuilt dist/
  tests/                  31 pytest tests
  docs/                   this documentation + submission deck
  DOCUMENTATION.md  DEMO_SCRIPT.md  PROJECT_STATUS.md  README.md
  run.ps1  run.sh  Dockerfile  docker-compose.yml  requirements.txt
```

Prahari — the sentinel for privileged access. Detect with AI and rules, explain every decision, respond in real time, and keep the proof quantum-safe.